

PLP-Lab-Aula 7 - Material Complementar

EXERCÍCIO 1 - Um número inteiro positivo é chamado de **número feliz** se, ao substituir o número pela **soma dos quadrados de seus dígitos** repetidamente, o processo eventualmente resultar no número 1. Caso contrário, o número é considerado **infeliz**, pois entra em um ciclo que não inclui o 1.

Exemplos:

- 19 é um número feliz:

$$1^2 + 9^2 = 82$$

$$8^2 + 2^2 = 68$$

$$6^2 + 8^2 = 100$$

$$1^2 + 0^2 + 0^2 = 1 \quad \checkmark$$

- 17 não é um número feliz, pois entra em um ciclo infinito sem chegar a 1 ✗

Escreva um código em Rust para responder se um número é ou não feliz.

RESPOSTA:

```
use std::collections::HashSet;
```

```
// Importa HashSet, que armazena valores únicos. Vamos usá-lo para detectar ciclos nos números.
```

```
use std::io;
```

```
// Importa o módulo para entrada de dados via teclado.
```

```
// Função que calcula a soma dos quadrados dos dígitos de um número.
```

```
// Exemplo: 82 ->  $8^2 + 2^2 = 64 + 4 = 68$ 
```

```
fn soma_dos_quadrados(mut n: i32) -> i32 {
```

```
    let mut soma = 0; // Inicializa a soma em 0
```

```
    while n > 0 {
```

```
        let digito = n % 10; // Pega o último dígito
```

```
        soma += digito * digito; // Soma o quadrado do dígito
```

```
        n /= 10; // Remove o último dígito
```

```
    }
```

```
    soma // Retorna o resultado final da soma dos quadrados
```

```
}
```

```

// Função que verifica se um número é feliz ou não

fn numero_feliz(mut n: i32) -> bool {

    let mut visitados = HashSet::new(); // Armazena os números já vistos para evitar
    ciclos

    // Continua enquanto n não for 1 e ainda não tenha sido visitado
    while n != 1 && !visitados.contains(&n) {

        visitados.insert(n); // Adiciona o número ao conjunto

        n = soma_dos_quadrados(n); // Calcula a próxima soma

    }

    n == 1 // Retorna true se chegou a 1, ou false se entrou em ciclo
}

fn main() {

    println!("Digite um número inteiro positivo:");

    let mut entrada = String::new(); // Armazena a entrada do usuário
    io::stdin()

        .read_line(&mut entrada) // Lê a linha digitada

        .expect("Erro ao ler entrada"); // Mostra erro se a leitura falhar

    // Tenta converter a entrada para inteiro

    let numero: i32 = match entrada.trim().parse() {

        Ok(n) if n > 0 => n, // Se for um número positivo válido, usamos ele

        _ => {

            println!("Por favor, digite um número inteiro positivo válido.");

            return; // Encerra se a entrada for inválida

        }

    };

    // Verifica se o número é feliz e imprime o resultado

```

```

if numero_feliz(numero) {

    println!("{}", "é um número feliz! 🐱", numero);

} else {

    println!("{}", "não é um número feliz. 🐼", numero);

}

}

```

Observações:

I)

1. `entrada.trim().parse()` tenta converter a entrada para um número.
2. O `match` verifica:
 - Se for `ok(n)` e `n > 0`, ele usa esse número.
 - Qualquer outro caso (como erro de parse ou número negativo) é capturado por `_`, e ele mostra a mensagem de erro e encerra o programa com `return`.

II)

Parte	Significado
<code>fn</code>	Declara uma função em Rust.
<code>soma_dos_quadrados</code>	Nome da função (pode ser chamado em outras partes do código).
<code>mut n: i32</code>	A função recebe um parâmetro chamado <code>n</code> , do tipo inteiro de 32 bits (<code>i32</code>), e indica que <code>n</code> é mutável dentro da função.
<code>-> i32</code>	A função retorna um valor do tipo <code>i32</code> (um número inteiro de 32 bits).

EXERCÍCIO 2 - Escreva um programa em Rust que solicite ao usuário um número inteiro positivo e verifique se ele é um **número primo**. Um número é considerado primo se for **maior que 1** e **divisível apenas por 1 e por ele mesmo**.

O programa deve exibir uma mensagem informando se o número é primo ou não. Escreva o código correspondente abaixo:

RESPOSTA

`use std::io;`

`// Importa o módulo para entrada de dados do teclado`

```

fn eh_primo(n: u32) -> bool {
    if n <= 1 {
        return false; // Números menores ou iguais a 1 não são primos
    }
    if n == 2 {
        return true; // 2 é o único número primo par
    }
    if n % 2 == 0 {
        return false; // Elimina os pares maiores que 2
    }

    let limite = (n as f64).sqrt() as u32; // Verifica até a raiz quadrada de n
    for i in 3..=limite {
        if n % i == 0 {
            return false; // Se for divisível por qualquer número além de 1 e ele mesmo, não
            é primo
        }
    }

    true // Se passou por tudo, é primo
}

fn main() {
    println!("Digite um número inteiro positivo:");

    let mut entrada = String::new();

    io::stdin()
        .read_line(&mut entrada)
        .expect("Erro ao ler entrada");

```

```

// Tenta converter a entrada para número inteiro positivo

let numero: u32 = match entrada.trim().parse() {
    Ok(n) if n > 0 => n,
    _ => {
        println!("Por favor, digite um número inteiro positivo válido.");
        return;
    }
};

// Chama a função e mostra o resultado

if eh_primo(numero) {
    println!("{}", "é um número primo! ☒", numero);
} else {
    println!("{}", "não é um número primo. ☐", numero);
}
}

```

EXERCÍCIO 3 – Escreva um programa em Rust que solicite ao usuário um número inteiro positivo e calcule o fatorial desse número. O fatorial de um número n ($n!$) é definido como o produto de todos os inteiros positivos de 1 até n .

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

O programa deve exibir o resultado do fatorial.

Exemplo de Entrada e Saída:

Digite um número inteiro positivo: 5

O fatorial de 5 é: 120

RESPOSTA

```
use std::io;
```

```
// Importa o módulo para entrada de dados via teclado
```

```

// Função que calcula o fatorial de um número usando laço iterativo
fn fatorial(n: u64) -> u64 {
    let mut resultado = 1;
    for i in 2..=n {
        resultado *= i; // Multiplica resultado por i a cada passo
    }
    resultado
}

fn main() {
    println!("Digite um número inteiro positivo:");

    let mut entrada = String::new(); // Cria uma string para armazenar a entrada

    io::stdin()
        .read_line(&mut entrada)
        .expect("Erro ao ler entrada");

    // Tenta converter a entrada para número inteiro não negativo (u64)
    let numero: u64 = match entrada.trim().parse() {
        Ok(n) => n,
        Err(_) => {
            println!("Por favor, digite um número inteiro positivo válido.");
            return;
        }
    };

    // Calcula o fatorial e exibe o resultado
    let resultado = fatorial(numero);
    println!("O fatorial de {} é {}.", numero, resultado);
}

```

EXERCÍCIO 4 – Escreva um programa em Rust que solicite ao usuário uma temperatura em graus Celsius e a converta para Fahrenheit e Kelvin.

As fórmulas de conversão são:

$$F = (C \times 9/5) + 32$$

$$K = C + 273.15$$

O programa deve exibir os valores convertidos com duas casas decimais.

Exemplo de Entrada e Saída:

Digite a temperatura em Celsius: 25

Em Fahrenheit: 77.00

Em Kelvin: 298.15

RESPOSTA

```
use std::io;
```

```
// Importa o módulo para entrada de dados via teclado
```

```
// Função que converte Celsius para Fahrenheit
```

```
fn celsius_para_fahrenheit(celsius: f64) -> f64 {  
    (celsius * 9.0 / 5.0) + 32.0  
}
```

```
// Função que converte Celsius para Kelvin
```

```
fn celsius_para_kelvin(celsius: f64) -> f64 {  
    celsius + 273.15  
}
```

```
fn main() {
```

```
    println!("Digite a temperatura em graus Celsius:");
```

```
    let mut entrada = String::new(); // Cria uma variável para armazenar a entrada
```

```
    io::stdin()
```

```

        .read_line(&mut entrada)

        .expect("Erro ao ler entrada"); // Lê a linha digitada

// Tenta converter a entrada para número decimal (f64)
let celsius: f64 = match entrada.trim().parse() {
    Ok(n) => n,
    Err(_) => {
        println!("Por favor, digite um número válido.");
        return;
    }
};

// Converte a temperatura para Fahrenheit e Kelvin
let fahrenheit = celsius_para_fahrenheit(celsius);
let kelvin = celsius_para_kelvin(celsius);

// Exibe os resultados
println!("{:.2}°C é equivalente a {:.2}°F e {:.2}K", celsius, fahrenheit, kelvin);
}

```

EXERCÍCIO 5 – Escreva um programa em Rust para somar duas matrizes de ordem 3.

RESPOSTA

```

use std::io;

// Importa o módulo para entrada de dados via teclado

// Função que soma duas matrizes 3x3
fn soma_matrizes(m1: [[i32; 3]; 3], m2: [[i32; 3]; 3]) -> [[i32; 3]; 3] {
    let mut resultado = [[0; 3]; 3]; // Inicializa a matriz resultado com zeros

    for i in 0..3 {
        for j in 0..3 {
            resultado[i][j] = m1[i][j] + m2[i][j]; // Soma os elementos correspondentes
        }
    }
}

```



```

    }
}

matriz // Retorna a matriz resultado
}

// Função para ler uma matriz 3x3 do usuário
fn ler_matriz() -> [[i32; 3]; 3] {

    let mut matriz = [[0; 3]; 3]; // Inicializa a matriz com zeros

    for i in 0..3 {
        for j in 0..3 {
            println!("Digite o elemento da matriz na posição [{}][{}]:", i + 1, j + 1);

            let mut entrada = String::new(); // Cria uma variável para armazenar a entrada
            io::stdin()
                .read_line(&mut entrada)
                .expect("Erro ao ler entrada"); // Lê a linha digitada
            matriz[i][j] = match entrada.trim().parse() {
                Ok(n) => n, // Converte a entrada para um número inteiro
                Err(_) => {
                    println!("Por favor, digite um número válido.");
                    return [[0; 3]; 3]; // Retorna uma matriz com zeros em caso de erro
                }
            };
        }
    }

    matriz // Retorna a matriz preenchida
}

fn main() {

```

```
println!("Digite os elementos da primeira matriz 3x3:");

let matriz1 = ler_matriz(); // Lê a primeira matriz


println!("Digite os elementos da segunda matriz 3x3:");

let matriz2 = ler_matriz(); // Lê a segunda matriz


// Soma as duas matrizes

let resultado = soma_matrizes(matriz1, matriz2);


// Exibe a matriz resultante

println!("A matriz resultante da soma é:");

for i in 0..3 {

    for j in 0..3 {

        print!("{}", resultado[i][j]);

    }

    println(); // Nova linha após cada linha da matriz

}

}
```

EXERCÍCIO 6 – Verificador de Palíndromo

Crie um programa em Rust que solicite ao usuário que digite uma **palavra ou frase**. O programa deve então verificar se o texto informado é um **palíndromo**.

Uma palavra/frase é considerada **palíndromo** se pode ser lida da mesma forma da esquerda para a direita e da direita para a esquerda, **desconsiderando espaços, acentuação e diferenças entre maiúsculas e minúsculas**.

Exemplo: arara, ovo, osso, radar, etc.

O programa deve exibir uma mensagem informando se o texto é ou não um palíndromo.

Exemplos de uso:

1. Entrada: Ame a ema

Saída: "Ame a ema" é um palíndromo! 

2. Entrada: Socorram-me subi no ônibus em Marrocos

Saída: "Socorram-me subi no ônibus em Marrocos" é um palíndromo! 

3. Entrada: OpenAI

Saída: "OpenAI" não é um palíndromo. 

RESPOSTA

```
use std::io;
```

```
fn limpar_string(s: &str) -> String {  
    s.chars()  
        .filter(|c| c.is_alphanumeric()) // Mantém apenas letras e números  
        .map(|c| c.to_ascii_lowercase()) // Converte para minúsculo  
        .collect()  
}
```

```
fn eh_palindromo(s: &str) -> bool {  
    let texto_limpo = limpar_string(s);  
    texto_limpo == texto_limpo.chars().rev().collect::<String>()  
}
```

```
fn main() {  
    println!("Digite uma palavra ou frase para verificar se é um palíndromo:");  
  
    let mut entrada = String::new();  
    io::stdin()  
        .read_line(&mut entrada)  
        .expect("Erro ao ler entrada");  
  
    let entrada = entrada.trim(); // Remove espaços extras do começo/fim  
    if eh_palindromo(entrada) {  
        println!("\"{}\" é um palíndromo! ☒, entrada);  
    } else {  
        println!("\"{}\" não é um palíndromo. ☐, entrada);  
    }  
}
```